

Software Design Document

for

Startup Incubator Management System

CONTENT

Contents	2
1. PURPOSE	3
1.1 Document Objectives	3
1.2 Intended Audience and Document Overview	3
1.3 Definitions, Acronyms and Abbreviations	3
1.4 References	4
2. ASSUMPTIONS AND CONSTRAINTS	4
2.1 Assumptions	4
2.2 Constraints	4
3. DATABASE-WIDE DESIGN DECISIONS	5
3.1 Behaviour	5
3.1.1 Login or Register	5
3.1.2 Startup Representative	5
3.1.3 Investor	5
3.1.4 Admin	5
3.1.5 Incubator Management	5
3.2 DBMS Platform	5
3.3 Security Requirements	6
3.4 Performance and Availability Decisions	6
4. DATABASE ADMINISTRATIVE FUNCTIONS	7
4.1 Entity-Relation Model	7
4.2 Relational Schema	8
4.3 Normalization	12
Appendix – SQL Schema (DDL)	12

1 Purpose

This Software Design Document (SDD) for the Startup Incubator Management System (SIMS) establishes a target database management system identified from the analysis of the software requirements, maintaining data consistency and integrity. The Entity-Relational model created by analysing the use case diagram is converted to a relational schema of the target Database Management System (DBMS).

1.1 Document Objectives

The Software Design Document has the following objectives:

1. To outline the software design and specification of the SIMS database, in addition to the system architecture and components that can be accessed by users or system developers via a DBMS.
2. To provide a fundamental approach for implementing the database and related software units, thus aiding in extracting details necessary for the software development of the application.

1.2 Intended Audience and Document Overview

- This document is intended for all stakeholders involved in the development, deployment, and use of the Startup Incubator Management System (SIMS). The primary audience includes system developers, database designers, project managers, and the course instructor. Additionally, it is useful for end users such as incubator management staff, system administrators, startup representatives, and investors who will interact with the system.
- The document provides a structured and detailed description of the system requirements. It is organized into multiple chapters covering the overall system description, functional and non-functional requirements, external interface specifications, and use case models. This SRS serves as a reference guide throughout the system development life cycle, helping ensure a clear understanding of system functionality, constraints, and expected behavior among all stakeholders.

1.3 Definitions, Acronyms and Abbreviations

S.No	Abbreviation / Term	Definition(s)
1	SIMS	Startup Incubator Management System
2	SDD	Software Design Document
3	SRS	Software Requirements Specification
4	DBMS	Database Management System
5	Admin	System Administrator with full database access
6	Startup	An early-stage company enrolled in the incubator program
7	Investor	Individual or entity providing funding to startups
8	KPI	Key Performance Indicator
9	MOU	Memorandum of Understanding
10	API	Application Programming Interface
11	RBAC	Role-Based Access Control
12	PK	Primary Key
13	FK	Foreign Key

S.No	Abbreviation / Term	Definition(s)
14	1NF	First Normal Form
15	2NF	Second Normal Form
16	3NF	Third Normal Form

1.4 References

- Software Requirements Specification – SIMS v1.0 (Group 08, 14/03/2026)
- Fundamentals of Database Systems by Ramez Elmasri and Shamkant Navathe
- IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications
- IEEE SRS Template – <http://www.cse.msu.edu/~cse870/IEEEExplore-SRS-template.pdf>
- Flask Documentation – <https://flask.palletsprojects.com>
- React.js Documentation – <https://react.dev>

2 Assumptions and Constraints

2.1 Assumptions

The following assumptions are made while developing this product:

- All startup, investor, funding, and milestone data must be stored in a relational database.
- The system must have sufficient storage capacity and provide fast access to the database.
- The system will be available 24/7 with minimal downtime.
- Users are assumed to have basic digital literacy; the interface will be entirely in English.
- The system administrator must have a working knowledge of incubator operations.
- The database must be updated upon every transaction – registration, funding, milestone update, etc.
- Since the application is web-based, users will possess a standard internet browser and decent connectivity.
- Users will have adequate knowledge of incubator operations

2.2 Constraints

The following design and implementation constraints are employed in the system:

- The system must be accessible through standard web browsers across multiple devices (desktop, laptop, tablet, mobile).
- Users must authenticate with valid credentials before accessing any system features.
- Role-based access control must be enforced for all users.
- All data updates (startup details, funding, etc.) must be reflected in real time in the database.
- Financial data must be stored with high precision and accuracy.
- The system must comply with data protection and privacy regulations.
- The implementation is limited to the chosen technology stack (MySQL, Flask, React.js).
- The system performance depends on internet connectivity and server availability.
- Only authorized administrators can perform critical operations like adding, updating, or deleting records.

3 Database-Wide Design Decisions

3.1 Behaviour

3.1.1 Login or Register

On opening the application, all users are directed to a login page. A registered user enters their email and password to log in. The system detects the user's role and redirects them to the appropriate role-specific dashboard. If the user does not have an account, they can register by providing their details and selecting their role (Startup Representative or Investor). After successful registration, they are directed to the appropriate dashboard.

3.1.2 Startup Representative

Startup Representatives, after logging in, are presented with the following options:

- View and update their startup's profile, including team details, domain, and description.
- Track milestones – view status (Pending, In Progress, Completed, Overdue) and completion dates.
- View funding history – all funding rounds received, equity given, and MOU statuses.
- View in-app notifications for upcoming review meetings, milestone due dates, and funding deadlines.

3.1.3 Investor

Investors, after logging in, are presented with the following options:

- Browse all registered startups and filter by domain, funding stage, and status.
- Express interest in a startup to initiate the investment discussion pipeline.
- View and manage their investment portfolio – equity held, amounts invested, and returns data.
- Receive in-app notifications for investment-related events.

3.1.4 System Auditing and Logging:

- The system shall maintain a complete activity log of all significant changes (e.g., profile updates, funding rounds, data deletions) for auditing purposes.
- Admins shall have access to the audit trail to track and verify system interactions.

3.1.5 Incubator Management

Incubator Management users, once signed in, can access:

- KPI dashboards showing aggregate statistics (total startups, total funding raised, graduation rate, etc.).
- Cohort overview tables and individual startup performance data.

3.2 DBMS Platform

SIMS is a web application that provides users with a clear and interactive experience. The design follows a three-tier architecture:

- Frontend: React.js with HTML5 and CSS3 – for building interactive, component-driven user interfaces. All interfaces follow a standard template and are accessible via any modern web browser.
- Backend: Flask (Python) – a lightweight web framework for building RESTful APIs that implement business logic, authentication, and RBAC enforcement.
- Database: MySQL – an open-source relational DBMS used to manage and organise all data in the SIMS.

The application allows users to log in with corresponding credentials and is directed to different dashboards according to their roles. The functionalities extended to different users differ in accordance with their roles.

3.3 Security Requirements

The system will store all data in a secure MySQL database. Users will be able to view information relevant to their role but will not have the privilege to modify data outside their permissions. This privilege is given to the Admin (super-admin), who alone has the right to perform full CRUD operations. These are the distinct types of accessors, each with varying access constraints enforced by RBAC middleware. Data Encryption will be used to secure sensitive data both in transit and at rest.

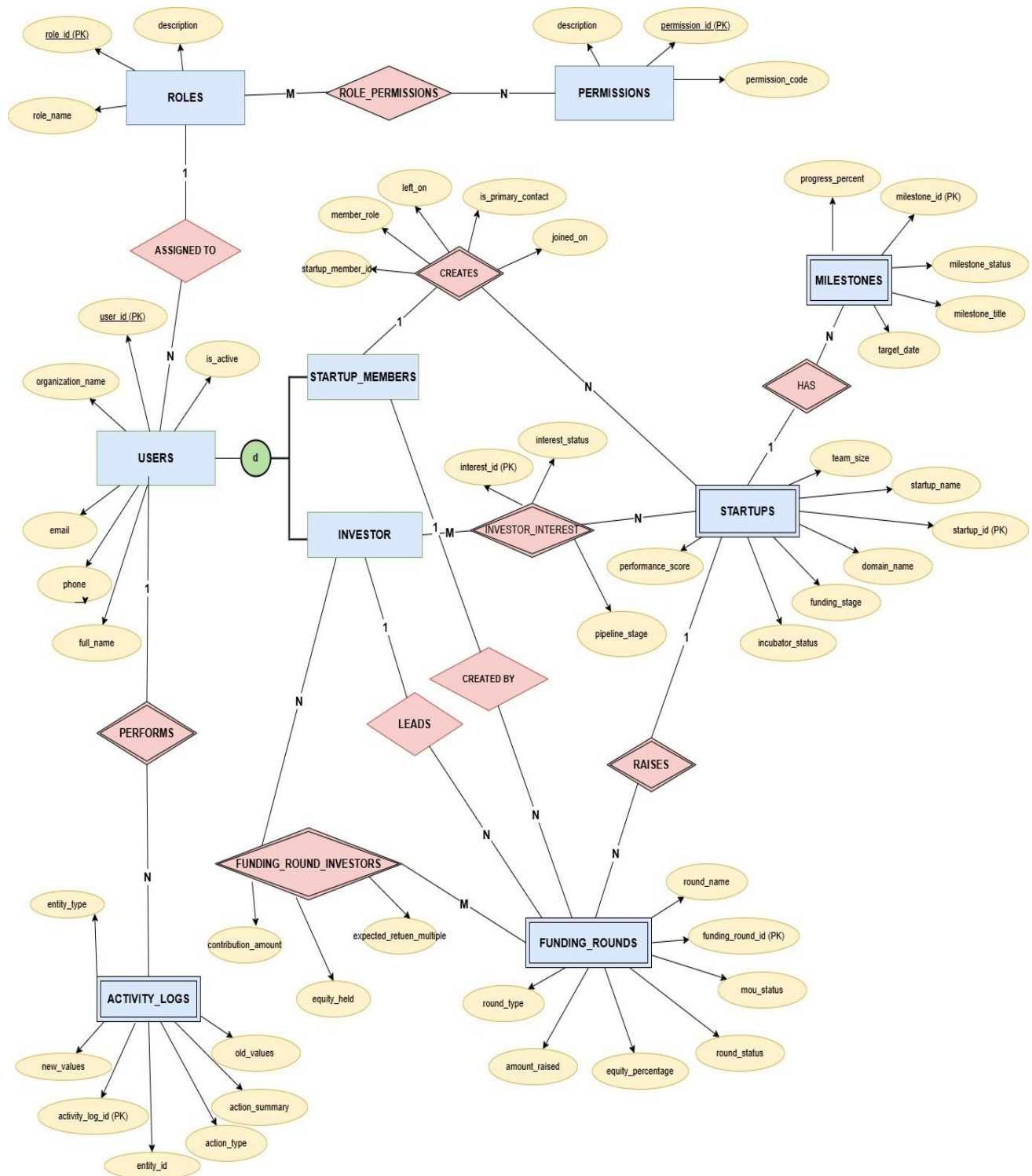
3.4 Performance and Availability Decisions

- The system shall provide fast response time for all user operations such as login, search, and data retrieval.
- Most user requests (e.g., viewing dashboards, browsing startups) should be processed within a few seconds under normal load conditions.
- The system shall support multiple concurrent users without significant degradation in performance.
- Database queries shall be optimized to ensure efficient data access and minimal latency.
- The system shall be available 24/7 with minimal downtime for maintenance.
- In case of server failure, backup and recovery mechanisms shall ensure quick restoration of services.
- The application shall handle peak loads (e.g., multiple investors accessing data simultaneously) efficiently.
- Regular monitoring and logging shall be implemented to track system performance and detect issues early.
- The system shall maintain at least 99% uptime to ensure high availability.
- A stable internet connection is required to ensure smooth system access and operation.

4 Database Administrative Functions

4.1 Entity-Relation Model

The following entities and relationships form the conceptual model of SIMS. Primary key attributes are underlined. The ER model captures all actors and their interactions within the incubator ecosystem.



4.2 Relational Schema

The following tables describe the complete schema with exact column names, data types, and constraints for each relation in the SIMS database. The schema is presented below with primary keys (PK), foreign keys (FK) noted.

roles

Column Name	Data Type	Constraints
role_id (PK)	INT	AUTO_INCREMENT, NOT NULL
role_name	VARCHAR(30)	NOT NULL, UNIQUE
description	VARCHAR(255)	NULL

permissions

Column Name	Data Type	Constraints
permission_id (PK)	INT	AUTO_INCREMENT, NOT NULL
permission_code	VARCHAR(100)	NOT NULL, UNIQUE
description	VARCHAR(255)	NOT NULL

role_permissions

Column Name	Data Type	Constraints
role_id (PK, FK)	INT	NOT NULL, FK→roles.role_id
permission_id (PK, FK)	INT	NOT NULL, FK→permissions.permission_id

users

Column Name	Data Type	Constraints
user_id (PK)	INT	AUTO_INCREMENT, NOT NULL
role_id (FK)	INT	NOT NULL, FK→roles.role_id
full_name	VARCHAR(120)	NOT NULL
email	VARCHAR(150)	NOT NULL, UNIQUE
phone	VARCHAR(20)	NULL
password_hash	VARCHAR(255)	NOT NULL
organization_name	VARCHAR(150)	NULL
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()

startups

Column Name	Data Type	Constraints
startup_id (PK)	INT	AUTO INCREMENT, NOT NULL
startup_name	VARCHAR(150)	NOT NULL, UNIQUE
domain_name	VARCHAR(100)	NOT NULL
funding_stage	VARCHAR(50)	NOT NULL
founding_date	DATE	NOT NULL
team_size	INT	NOT NULL, CHECK(≥ 0)
headquarters_city	VARCHAR(100)	NULL
website_url	VARCHAR(255)	NULL
description	TEXT	NULL
incubator_status	VARCHAR(30)	NOT NULL, DEFAULT 'ACTIVE'
funding_status	VARCHAR(50)	NOT NULL, DEFAULT 'BOOTSTRAPPED'
performance_score	DECIMAL(5,2)	NULL
created_by (FK)	INT	NOT NULL, FK→users.user_id
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()
updated_at	TIMESTAMP	NOT NULL, DEFAULT NOW()

creates

Column Name	Data Type	Constraints
startup_member_id (PK)	INT	AUTO INCREMENT, NOT NULL
startup_id (FK)	INT	NOT NULL, FK→startups.startup_id
user_id (FK)	INT	NOT NULL, FK→users.user_id
member_role	VARCHAR(50)	NOT NULL
joined_on	DATE	NOT NULL, DEFAULT CURRENT_DATE
left_on	DATE	NULL
is_primary_contact	BOOLEAN	NOT NULL, DEFAULT FALSE

milestones

Column Name	Data Type	Constraints
milestone_id (PK)	INT	AUTO_INCREMENT, NOT NULL
startup_id (FK)	INT	NOT NULL, FK→startups.startup_id
milestone_title	VARCHAR(150)	NOT NULL
milestone_description	TEXT	NULL
target_date	DATE	NULL
completion_date	DATE	NULL
milestone_status	VARCHAR(30)	NOT NULL, DEFAULT 'PENDING'
progress_percent	DECIMAL(5,2)	NOT NULL, DEFAULT 0, CHECK(0–100)

investor interests

Column Name	Data Type	Constraints
interest_id (PK)	INT	AUTO_INCREMENT, NOT NULL
startup_id (FK)	INT	NOT NULL, FK→startups.startup_id
investor_id (FK)	INT	NOT NULL, FK→users.user_id
interest_status	VARCHAR(30)	NOT NULL, DEFAULT 'INITIATED'
pipeline_stage	VARCHAR(50)	NOT NULL, DEFAULT 'INTRODUCTION'
notes	TEXT	NULL
expressed_at	TIMESTAMP	NOT NULL, DEFAULT NOW()

funding rounds

Column Name	Data Type	Constraints
funding_round_id (PK)	INT	AUTO_INCREMENT, NOT NULL
startup_id (FK)	INT	NOT NULL, FK→startups.startup_id
round_name	VARCHAR(100)	NOT NULL
round_type	VARCHAR(50)	NOT NULL
amount_raised	DECIMAL(15,2)	NOT NULL, CHECK(≥0)

Column Name	Data Type	Constraints
equity_percentage	DECIMAL(5,2)	NOT NULL, CHECK(0–100)
lead_investor_id (FK)	INT	NULL, FK→users.user_id
mou_status	VARCHAR(30)	NOT NULL, DEFAULT 'PENDING'
round_status	VARCHAR(30)	NOT NULL, DEFAULT 'OPEN'
announced_on	DATE	NULL
closed_on	DATE	NULL
created_by (FK)	INT	NOT NULL, FK→users.user_id

funding round investors

Column Name	Data Type	Constraints
funding_round_id (PK, FK)	INT	NOT NULL, FK→funding_rounds.funding_round_id
investor_id (PK, FK)	INT	NOT NULL, FK→users.user_id
contribution_amount	DECIMAL(15,2)	NOT NULL, CHECK(≥0)
equity_held	DECIMAL(5,2)	NOT NULL, CHECK(0–100)
expected_return_multiple	DECIMAL(8,2)	NULL

activity logs

Column Name	Data Type	Constraints
activity_log_id (PK)	INT	AUTO_INCREMENT, NOT NULL
actor_user_id (FK)	INT	NOT NULL, FK→users.user_id
entity_type	VARCHAR(50)	NOT NULL
entity_id	INT	NOT NULL
action_type	VARCHAR(30)	NOT NULL
action_summary	VARCHAR(255)	NOT NULL
old_values	TEXT	NULL
new_values	TEXT	NULL
created_at	TIMESTAMP	NOT NULL, DEFAULT NOW()

4.3 Normalization

1NF – The tables are in 1NF, as there are no multivalued or composite attributes. Each table contains atomic values, and repeating groups have been eliminated by creating separate tables such as startup_members and

funding_round_investors. Each record is uniquely identified by its primary key. Hence, the database is in 1NF.

- **2NF** – The tables are already in 1NF as proved above. There are no partial dependencies present in the schema. All non-prime attributes are fully functionally dependent on the entire primary key. The composite-key tables such as role_permissions, startup_members, funding_round_investors contain attributes that depend on the complete composite key rather than a part of it. Hence, the database is in 2NF.

- **3NF** – The tables are already in 2NF as proved above. There are no transitive functional dependencies in any of the tables. Non-prime attributes do not depend on other non-prime attributes. For example, in the funding_rounds table, attributes like amount_raised and equity_percentage depend only on funding_round_id and not on any other attribute. Similarly, user role details are stored separately in the roles table, avoiding redundancy. Hence, the database is in 3NF.

Appendix – SQL Schema (DDL)

The following Data Definition Language (DDL) script creates the complete SIMS relational database schema in MySQL. All tables, primary keys, foreign keys, check constraints, and default values are included.

```
CREATE TABLE roles (  
    role_id INT PRIMARY KEY AUTO_INCREMENT,  
    role_name VARCHAR(30) NOT NULL UNIQUE,  
    description VARCHAR(255)  
);  
  
CREATE TABLE permissions (  
    permission_id INT PRIMARY KEY AUTO_INCREMENT,  
    permission_code VARCHAR(100) NOT NULL UNIQUE,  
    description VARCHAR(255) NOT NULL  
);  
  
CREATE TABLE role_permissions (  
    role_id INT NOT NULL,  
    permission_id INT NOT NULL,  
    PRIMARY KEY (role_id, permission_id),  
    FOREIGN KEY (role_id) REFERENCES roles(role_id),  
    FOREIGN KEY (permission_id) REFERENCES permissions(permission_id)  
);  
  
CREATE TABLE users (  
    user_id INT PRIMARY KEY AUTO_INCREMENT,  
    role_id INT NOT NULL,  
    full_name VARCHAR(120) NOT NULL,  
    email VARCHAR(150) NOT NULL UNIQUE,  
    phone VARCHAR(20),  
    password_hash VARCHAR(255) NOT NULL,  
    organization_name VARCHAR(150),  
    is_active BOOLEAN NOT NULL DEFAULT TRUE,  
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    FOREIGN KEY (role_id) REFERENCES roles(role_id)  
);
```

```
CREATE TABLE startups (  
  startup_id INT PRIMARY KEY AUTO_INCREMENT,  
  startup_name VARCHAR(150) NOT NULL UNIQUE,  
  domain_name VARCHAR(100) NOT NULL,  
  funding_stage VARCHAR(50) NOT NULL,  
  founding_date DATE NOT NULL,  
  team_size INT NOT NULL CHECK (team_size >= 0),  
  headquarters_city VARCHAR(100),  
  website_url VARCHAR(255),  
  description TEXT,  
  incubator_status VARCHAR(30) NOT NULL DEFAULT 'ACTIVE',  
  funding_status VARCHAR(50) NOT NULL DEFAULT 'BOOTSTRAPPED',  
  performance_score DECIMAL(5,2),  
  created_by INT NOT NULL,  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (created_by) REFERENCES users(user_id)  
);  
  
CREATE TABLE creates (  
  startup_member_id INT PRIMARY KEY AUTO_INCREMENT,  
  startup_id INT NOT NULL,  
  user_id INT NOT NULL,  
  member_role VARCHAR(50) NOT NULL,  
  joined_on DATE NOT NULL DEFAULT CURRENT_DATE,  
  left_on DATE,  
  is_primary_contact BOOLEAN NOT NULL DEFAULT FALSE,  
  UNIQUE (startup_id, user_id),  
  FOREIGN KEY (startup_id) REFERENCES startups(startup_id),  
  FOREIGN KEY (user_id) REFERENCES users(user_id)  
);  
  
CREATE TABLE milestones (  
  milestone_id INT PRIMARY KEY AUTO_INCREMENT,  
  startup_id INT NOT NULL,  
  milestone_title VARCHAR(150) NOT NULL,  
  milestone_description TEXT,  
  target_date DATE,  
  completion_date DATE,  
  milestone_status VARCHAR(30) NOT NULL DEFAULT 'PENDING',  
  progress_percent DECIMAL(5,2) NOT NULL DEFAULT 0 CHECK (progress_percent BETWEEN 0  
AND 100),  
  FOREIGN KEY (startup_id) REFERENCES startups(startup_id)  
);  
  
CREATE TABLE investor_interests (  
  interest_id INT PRIMARY KEY AUTO_INCREMENT,  
  startup_id INT NOT NULL,  
  investor_id INT NOT NULL,  
  interest_status VARCHAR(30) NOT NULL DEFAULT 'INITIATED',  
  pipeline_stage VARCHAR(50) NOT NULL DEFAULT 'INTRODUCTION',  
  notes TEXT,  
  expressed_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  UNIQUE (startup_id, investor_id),  
  FOREIGN KEY (startup_id) REFERENCES startups(startup_id),  
  FOREIGN KEY (investor_id) REFERENCES users(user_id)  
);
```

```
CREATE TABLE funding_rounds (  
  funding_round_id INT PRIMARY KEY AUTO_INCREMENT,  
  startup_id INT NOT NULL,  
  round_name VARCHAR(100) NOT NULL,  
  round_type VARCHAR(50) NOT NULL,  
  amount_raised DECIMAL(15,2) NOT NULL CHECK (amount_raised >= 0),  
  equity_percentage DECIMAL(5,2) NOT NULL CHECK (equity_percentage BETWEEN 0 AND 100),  
  lead_investor_id INT,  
  mou_status VARCHAR(30) NOT NULL DEFAULT 'PENDING',  
  round_status VARCHAR(30) NOT NULL DEFAULT 'OPEN',  
  announced_on DATE,  
  closed_on DATE,  
  created_by INT NOT NULL,  
  FOREIGN KEY (startup_id) REFERENCES startups(startup_id),  
  FOREIGN KEY (lead_investor_id) REFERENCES users(user_id),  
  FOREIGN KEY (created_by) REFERENCES users(user_id)  
);
```

```
CREATE TABLE funding_round_investors (  
  funding_round_id INT NOT NULL,  
  investor_id INT NOT NULL,  
  contribution_amount DECIMAL(15,2) NOT NULL CHECK (contribution_amount >= 0),  
  equity_held DECIMAL(5,2) NOT NULL CHECK (equity_held BETWEEN 0 AND 100),  
  expected_return_multiple DECIMAL(8,2),  
  PRIMARY KEY (funding_round_id, investor_id),  
  FOREIGN KEY (funding_round_id) REFERENCES funding_rounds(funding_round_id),  
  FOREIGN KEY (investor_id) REFERENCES users(user_id)  
);
```

```
CREATE TABLE activity_logs (  
  activity_log_id INT PRIMARY KEY AUTO_INCREMENT,  
  actor_user_id INT NOT NULL,  
  entity_type VARCHAR(50) NOT NULL,  
  entity_id INT NOT NULL,  
  action_type VARCHAR(30) NOT NULL,  
  action_summary VARCHAR(255) NOT NULL,  
  old_values TEXT,  
  new_values TEXT,  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (actor_user_id) REFERENCES users(user_id)  
);
```